

CHOICE Forge — Layer 1 Explainer

What spaCy actually gives you: tokenization, POS tags, dependency relations, lemmas, and named entities — explained plainly, then technically, with code.

Worked example used throughout: "Get monthly logo churn under 2% for the SMB tier before the board meeting in November."

LAYER 1 • TERM 0

Tokenization (the step everything else depends on)

In plain terms

Breaking a sentence into its individual words and punctuation marks — like cutting a sentence into pieces with scissors along the spaces.

Technically

spaCy splits raw text into a **Doc** object: a sequence of **Token** objects. It's smarter than `.split(" ")` — it knows how to handle contractions, hyphenated terms, and punctuation correctly. Every other annotation (POS, dependency, lemma, entity) attaches to these tokens.

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Get monthly logo churn under 2% for the SMB tier before the board meeting in November.")
print([t.text for t in doc])

# ['Get', 'monthly', 'logo', 'churn', 'under', '2', '%', 'for', 'the',
#  'SMB', 'tier', 'before', 'the', 'board', 'meeting', 'in', 'November', '.']
```

Why it matters in this pipeline

Notice "2%" splits into two tokens, "2" and "%" — this matters later because it affects how the CRF's predicted spans get assembled back into text.

LAYER 1 • TERM 1

POS tag (part of speech)

In plain terms

Is this word a doing-word, a thing-word, a describing-word? Verb, noun, adjective, and so on — the grammar-class label from school.

Technically

spaCy gives two levels of granularity. `token.pos_` is the coarse universal tag (VERB, NOUN, ADJ, ADP, DET, NUM, PROP, PUNCT). `token.tag_` is fine-grained and model-specific (e.g. "VBG" for a

gerund-verb vs “VB” for base-form — both collapse to `pos_ == VERB`, but `tag_` keeps the distinction).

```
for tok in doc:
    print(f"{tok.text:10} pos={tok.pos_:6} tag={tok.tag_}")
```

```
# Get      pos=VERB   tag=VB
# monthly  pos=ADJ    tag=JJ
# logo     pos=NOUN   tag=NN
# churn    pos=NOUN   tag=NN
# under    pos=ADP    tag=IN
# 2        pos=NUM    tag=CD
# %        pos=NOUN   tag=NN
# for      pos=ADP    tag=IN
# SMB      pos=PROPN  tag=NNP
# tier      pos=NOUN   tag=NN
# November pos=PROPN  tag=NNP
```

Why it matters in this pipeline

This is what let the rule-based Layer 2 know “Get” is a VERB candidate for the root/intent, and it's one of the strongest CRF features in `train_crf.py`'s `token_features()` — nouns cluster into scope/object/measure spans, ADP (prepositions) mark span boundaries.

Dependency relation

In plain terms

Who's doing what to whom, grammatically. Not just “these are nouns and verbs” but “this noun is the thing being acted on by that verb,” “this phrase describes that other phrase.” It's the sentence's grammatical wiring diagram.

Technically

spaCy parses the sentence into a tree. Every token has a `dep_` label (its grammatical relation) and a `head` (the token it's attached to). The verb at the top with no parent gets `dep_ == "ROOT"`. Common labels: `nsubj` (subject), `dobj` (direct object), `prep` (preposition attaching a phrase), `pobj` (object of that preposition), `compound` (noun-noun modifier, like “logo” modifying “churn”), `amod` (adjective modifier).

```
for tok in doc:
    print(f"{tok.text:10} dep={tok.dep_:10} head={tok.head.text}")
```

```
# Get      dep=ROOT      head=Get
# monthly  dep=amod      head=churn
# logo     dep=compound  head=churn
# churn    dep=dobj      head=Get
# under    dep=prep      head=churn
# 2        dep=pobj      head=under
# for      dep=prep      head=Get
# SMB      dep=compound  head=tier
# tier      dep=pobj      head=for
# before   dep=prep      head=Get
# meeting  dep=pobj      head=before
```

Why it matters in this pipeline

The single most useful signal in the whole pipeline. Read it as a tree, not a list: “Get” is the root, “churn” hangs off it as the `dobj` (the **measure** field), “for → tier” hangs off “Get” as a prepositional phrase (the **scope** field), “under → 2” hangs off “churn” (the **magnitude** field). The original rule-based Layer 2 walked this tree by hand — “find the `dobj` of root, find prep phrases headed by ‘for’.” The CRF now learns the same pattern statistically, using exactly this `dep/head` information as input features (`"dep": tok.dep_, "head_dep": tok.head.dep_` in `token_features()`).

Lemma

In plain terms

The dictionary form of a word — “running,” “ran,” “runs” all point back to “run.” It strips away tense, plurals, and conjugation to get the base word.

Technically

`token.lemma_` gives the canonical form, computed from `pos_` plus a lookup/rule table (a plural noun and a past-tense verb are lemmatized differently even when they look similar on the surface).

```
onboard_doc = nlp("The team onboarded 1,200 stores.")
for tok in onboard_doc:
    print(tok.text, "->", tok.lemma_)
```

```
# The      -> the
# team     -> team
# onboarded -> onboard
# 1,200    -> 1,200
# stores   -> store
```

Why it matters in this pipeline

The earlier rule-based intent lookups keyed off `root.lemma_.lower()` rather than the raw surface word — that's why “onboarding,” “onboards,” and “onboarded” all match the same rule without writing three separate entries. Same reason `token_features()` includes `"lemma": tok.lemma_` and `"head_lemma": tok.head.lemma_` — it generalizes the CRF's learning across verb tenses instead of treating “increase” and “increasing” as two unrelated words.

Entity type (Named Entity Recognition / NER)

In plain terms

spaCy has already been trained to recognize certain kinds of real-world “things” in text — dates, percentages, money amounts, organization names, people’s names — and it tags the exact stretch of text that refers to one.

Technically

This operates at the **span** level, not the token level (an entity can span multiple tokens). `doc.ents` gives a list of Span objects, each with `.text` and `.label_`. Individual tokens inside an entity also get `token.ent_type_set`.

```
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
# 2%          -> PERCENT
# November    -> DATE
```

Why it matters in this pipeline

The one piece of Layer 1 that sometimes answers a field almost directly, rather than just feeding the CRF. This is exactly why `ent_type` is one of `token_features()`'s inputs, and why magnitude/time roles were the easiest for the CRF to learn (F1 0.77 on TIME): the NER model has already done most of the work, the CRF just has to learn “the token right before a DATE entity, headed by 'before' or 'by', gets tagged as TIME.”

Caveat: “SMB” wasn’t picked up as an entity here — spaCy’s default small model doesn’t know business jargon/abbreviations, it only recognizes the entity types it was trained on. Domain terms like “kirana stores” or “SMB tier” fall through NER entirely and rely on the dependency parse + the CRF’s learned patterns instead.

Quick reference

Concept	spaCy attribute	Granularity	Used for
Tokenization	<code>doc / Doc(...)</code>	whole text	foundation for everything else
POS tag	<code>token.pos_ / token.tag_</code>	per token	role-boundary signal, coarse + fine
Dependency relation	<code>token.dep_ / token.head</code>	per token (tree)	grammatical structure → role mapping
Lemma	<code>token.lemma_</code>	per token	generalizes across verb tense/plurals
Entity type	<code>doc.ents / token.ent_type_</code>	per span	dates, percentages, money — near-direct field answers